

1. Scale Strategy (500,000+ Records)

To scale this pipeline to handle 500,000+ Startups, Products, and Research Papers without manual intervention, the architecture is designed around decoupled, horizontally scalable micro-crawlers and event-driven data ingestion.

- **Distributed Crawling:** Instead of a monolithic single runner, each crawler (e.g., StartupCrawler, ProductCrawler) can be containerized via Docker and orchestrated via a serverless environment (e.g., AWS Fargate, GCP Cloud Run) to run concurrently.
- **Message Queues:** The crawlers fetch raw HTML/JSON and push the unstructured payloads into a message broker like Apache Kafka or AWS SQS.
- **Stateless Extraction Nodes:** A fleet of stateless worker nodes consumes the raw payloads from the queue, executes the LLM extraction (LLMOrchestrator), and performs Entity Resolution, pushing the final structured JSON into the storage layer. This allows the computationally heavy LLM extraction to scale independently of the I/O-bound web crawlers.

2. Handling 413s & 429s (Context Windows and Rate Limits)

Handling thousands of concurrent LLM extractions requires an aggressive, multi-tiered approach to prevent API exhaustion and payload crashes.

- **Intelligent Chunking (413 Payload Too Large):** Many AI research papers or heavily-texted news sites exceed the token context window. To guarantee we never trigger a 413 error while retaining the most semantically dense data, the LLMOrchestrator implements a strict 15,000 character truncation strategy. It dynamically slices out the middle of the text, securely retaining the first 7,500 and last 7,500 characters. Since the Abstract/Introduction and Conclusion contain the highest density of structured metadata (e.g. pricing, authors, core product loops), data fidelity remains incredibly high.
- **Provider Fallback Chain (429 Too Many Requests):** Rate limits are handled gracefully using a sophisticated fallback mechanism. The LLMOrchestrator is configured with a priority chain: Groq → Gemini → DeepSeek.
- **Exponential Backoff:** Using the tenacity library, each provider attempt is wrapped in an asynchronous retry loop with exponential backoff and jitter. If Groq triggers a 429, the system backs off dynamically. If Groq is completely exhausted, it instantly falls through to Gemini.

3. Freshness Tracking (Preventing Duplicates)

To ensure we never process the same article or job twice across distributed crawler nodes, the pipeline employs a deterministic hashing and idempotent upsert strategy.

- **Deterministic Content ID:** Every scraped entity undergoes a normalization pass on its URL. The normalized URL is hashed using SHA-256 to generate a globally unique content_id.
- **Intelligent Heuristic for Missing Dates:** For high-value sources that aggressively strip meta publication dates (e.g. AllJobs.net), the pipeline queries the central database for the content_id during ingestion. If the record exists, the pipeline forces the new incoming payload to inherit the original collected_at timestamp. This prevents stale, recycled job

listings from artificially resetting their "24-hour freshness" timer. If the content_id is missing, it is definitively flagged as a fresh, unseen signal.

- **Idempotent DB Upserts:** The storage repositories utilize native SELECT WHERE content_id upsert mechanics, guaranteeing that distributed race conditions do not result in duplicated rows.

4. Storage Strategy

- **Primary Database (SQLite -> PostgreSQL):** For this trial phase, SQLite with SQLAlchemy Async was chosen for its zero-configuration portability and native asynchronous driver support (aiosqlite). For production scaling to 500,000+ records, this will seamlessly migrate to PostgreSQL. Postgres natively supports JSONB columns, which is critical for storing the semi-structured nested content schemas while maintaining strict relational integrity for canonical entity matching.
- **Graph Storage (Neo4j):** As GraphOne's ultimate goal is building an "Intelligence Graph", relational databases fall short when querying multi-hop relationships (e.g., "Find all founders who worked at a startup funded by Sequoia, who are now authoring AI research papers"). Neo4j will be deployed alongside Postgres. Entities (Startups, People, Papers) will be mapped as Nodes, and relationships (FOUNDED, AUTHORED, MENTIONS) as Edges, allowing sub-millisecond traversal of the venture ecosystem.